

1 — Gateway (Passerelle API)

Définition :

- Une **API Gateway** est un point d'entrée unique pour tous les clients qui veulent accéder aux différents services backend (microservices).
- Elle fait **route, sécurité, agrégation et transformation des requêtes**.

Rôles principaux :

1. **Routage** : redirige les requêtes vers le bon microservice selon l'URL ou le header.
 - Exemple : `/auth/login` → service Auth ; `/api/users` → service User.
2. **Sécurité / authentication** : peut valider les JWT tokens, limiter le nombre de requêtes (rate limiting), filtrer les IP.
3. **Transformation** : peut modifier les headers, réponses, ou agréger plusieurs services en une seule réponse.
4. **Monitoring** : collecte métriques et logs centralisés pour observabilité.

Exemples populaires :

- **Spring Cloud Gateway** (Java/Spring Boot)
- **Kong, Traefik, NGINX**
- **API Gateway AWS / Azure / GCP**

Schéma simple :

Client Angular → API Gateway → Microservices (Auth, User, Product, etc.)

2 — Load Balancing (Répartition de charge)

Définition :

- Le **load balancer** distribue les requêtes entrantes sur plusieurs instances d'un service pour **équilibrer la charge** et améliorer **haute disponibilité**.

Types :

1. **Round Robin** : distribue à tour de rôle.
2. **Least Connections** : envoie la requête au serveur ayant le moins de connexions actives.
3. **IP Hash / Sticky Session** : maintient un client vers le même serveur si nécessaire.

Exemples de Load Balancers :

- Logiciel : Nginx, HAProxy
- Cloud : AWS ELB, Azure Load Balancer

Architecture simple :

Client → Load Balancer → [Service Instance 1, Service Instance 2, Service Instance 3]

- Si une instance tombe, le LB redirige automatiquement vers les instances disponibles.

3 — Root API

Définition :

- Le **root API** est l'URL de base pour accéder à un service backend.
- Exemple : si ton API produit est déployée sur `https://api.mysite.com/v1/products`, le **root API** = `https://api.mysite.com/v1/`.
- Tous les clients Angular ou autres applications consomment les endpoints relatifs à ce root.

Bonnes pratiques :

- Versionner les APIs (`v1`, `v2`) pour gérer les évolutions sans casser les clients.
 - Documenter via Swagger/OpenAPI pour que les développeurs frontend sachent comment consommer l'API.
-

4 — Client Angular

Rôle :

- L'**application Angular** est le frontend, qui fait des requêtes HTTP vers les APIs.
- Elle ne parle pas directement aux microservices, mais passe souvent par la **gateway** pour simplifier la communication et sécuriser l'accès.

Exemple pratique :

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private apiRoot = 'https://api.mysite.com/v1';

  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http.get(`${this.apiRoot}/users`);
  }
}
```

}

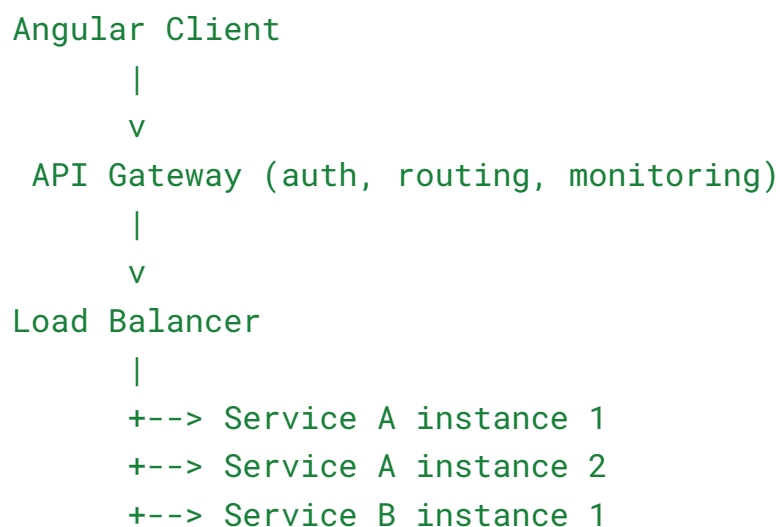
- Ici, `UserService` utilise `apiRoot` comme **root API**, et le Gateway load balance la requête vers une instance disponible du service User.

5 — Comment Gateway + Load Balancer + Angular interagissent

Workflow typique :

1. Client Angular fait une requête HTTP vers l'API root via Gateway.
2. Gateway décide quel service traiter, applique la sécurité et envoie la requête.
3. Si le service est répliqué sur plusieurs instances, **load balancer** choisit celle avec moins de charge ou selon la stratégie configurée.
4. Service répond → Gateway peut transformer ou agréger la réponse → renvoie au client Angular.

Schéma simple :



6 — Commandes pratiques pour tester et monitoring

Test API via Angular / curl :

```
curl -X GET https://api.mysite.com/v1/users -H "Authorization: Bearer <token>"
```

Vérifier disponibilité du service / load balancing :

```
# Vérifier les pods / instances (Kubernetes example)
kubectl get pods -n mynamespace
```

```
# Vérifier load balancer sur Nginx
curl -I https://gateway.mysite.com
```

Angular dev tools :

- Network tab → voir requêtes envoyées au Gateway
- Inspecter headers, status code, latency

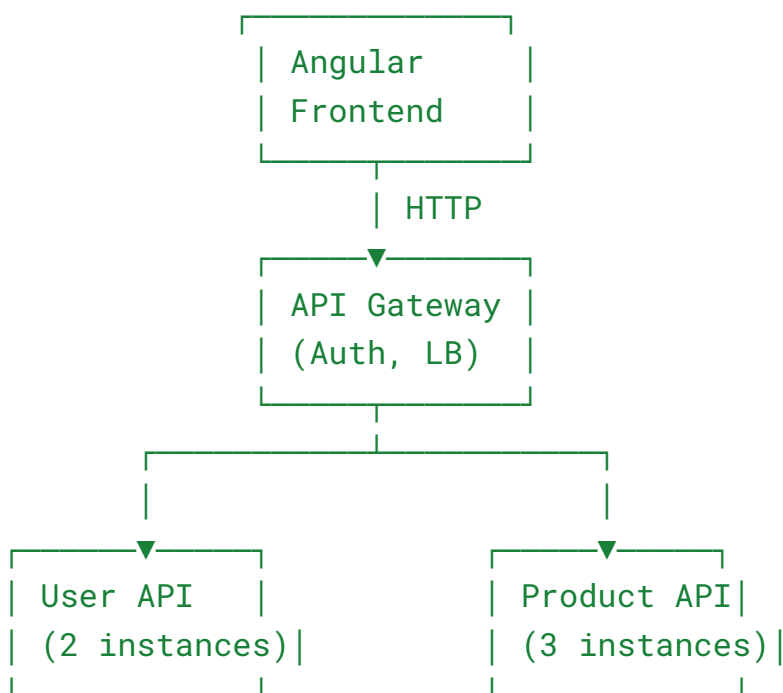
7 — Bonnes pratiques / points clés

1. **API Gateway = seul point d'entrée** → simplifie sécurité et monitoring.
2. **Load Balancer** → haute disponibilité + répartition charge.
3. **Root API** → versionnée et stable → Angular consomme uniquement ce root.
4. **Sécurité** : JWT, rate limiting, CORS.
5. **Logs et monitoring centralisés** pour diagnostiquer latence ou erreurs.

6. **Scalabilité horizontale** : plus d'instances services → Gateway et LB répartissent.
-

8 — Exemple d'architecture complète pour un projet réel

Supposons un projet *vprofil* microservices :



- Chaque service peut avoir plusieurs instances → Load Balancer interne à Gateway.
- Angular consomme uniquement `/v1/users` ou `/v1/products` → Gateway gère le reste.